

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

Escuela profesional de Ingeniería Informática y de Sistemas



Tema: Learnix_OS Gestión de procesos y sistema de archivos

Docente: Edwin Carrasco Poblete

Estudiantes:

- Mamani Condori Franklin Gilberto
- Tito Huaman Jack Eliezer

Semestre: 2026-I

Cusco – Perú 2026

Índice

Introducción	1
Gestión de Procesos y Cambio de Contexto (Tarea del 17 de julio)	1
Bloque de control de proceso (PCB)	1
Pila inicial de un proceso	2
Mecanismo de cambio de contexto	2
Resultado obtenido.....	3
Problemas resueltos	3
Planificador con Prioridades (Tarea del 24 de julio)	4
Diseño de las prioridades	4
Función scheduler_tick	4
Resultado Obtenido.....	5
Verificación.....	5
Driver de Disco y Sistema de Archivos FAT16 (Tarea del 31 de julio).....	6
Capas del sistema de archivos.....	7
Driver ATA en modo PIO.....	7
Decisión clave: selección del disco Slave.....	8
Estructura FAT16 del disco de 16 MB	8
Formato de nombres de archivo.....	8
Resultados de la verificación	9
Funciones de administración de archivos	9
Cargador de Ejecutables ELF (Tarea del 7 de agosto)	10

Estructuras del formato ELF	10
Proceso de carga	10
Resultado obtenido.....	11
Problema principal: segmentos fuera del área mapeada	11
Otro problema: orden de inicialización.....	13
Restricciones de los programas de usuario	13
Llamadas al Sistema y Shell (Tarea del 14 de agosto)	13
Mecanismo INT 0x80	13
Llamadas al sistema implementadas	15
Shell interactivo	15
Resultado obtenido.....	16
Problema 1: el teclado no respondía	16
Problema 2: todo se escribía en la esquina de la pantalla	17
Problema 3: ls y cat colgaban el sistema	17
Problema 4: las tareas de fondo bloqueaban el shell	17
Modelo de Entrada/Salida en Learnix OS	18
Resumen General de Problemas y Soluciones.....	18
Estado del Proyecto.....	19
Implementado	19
Pendiente.....	19
Conclusiones	20
Bibliografía	21

Índice de Tablas

Tabla . <i>Problemas resueltos durante la gestión de procesos y el cambio de contexto</i>	3
Tabla . <i>Niveles de prioridad y quantum asignado por el planificador</i>	4
Tabla . <i>Correspondencia entre las capas del sistema de archivos y las funciones que las implementan</i>	7
Tabla . <i>Funciones de administración de archivos implementadas en Learnix OS</i>	9
Tabla . <i>Convención de registros para invocar una llamada al sistema</i>	13
Tabla . <i>Llamadas al sistema implementadas en Learnix OS</i>	15
Tabla . <i>Mecanismos de entrada/salida y su estado de implementación</i>	18
Tabla . <i>Resumen cronológico de problemas de depuración y sus soluciones</i>	18

Índice de Figuras

Figura 1. <i>Diagrama de estados de un proceso en Learnix OS</i>	2
Figura 2: <i>Gestion de procesos con planificador Round Robin</i>	3
Figura 3: <i>Gestión y planificación de procesos</i>	5
Figura 4. <i>Reparto medido de tiempo de CPU entre tres tareas de distinta prioridad</i>	6
Figura 5. <i>Capas del sistema de gestión de archivos de Learnix OS</i>	6
Figura 6. <i>Distribución de regiones en el disco virtual FAT16 de 16 MB</i>	8
Figura 7 <i>Learnix en ejecución sobre ATA y sistema de archivos FAT 16</i>	9
Figura 8: <i>Lectura de archivos desde FAT 26</i>	11
Figura 9. <i>Mapa de memoria de 4 MB y el segmento generado fuera del área mapeada</i>	12
Figura 10. <i>Flujo de una llamada al sistema a través de INT 0x80</i>	14
Figura 11: <i>Visualizar desde el Shell con 7 comandos disponibles</i>	16

Introducción

El presente informe documenta el desarrollo de Learnix OS, un sistema operativo educativo de 32 bits para arquitectura x86 construido desde cero en C y ensamblador, correspondiente al curso de Sistemas Operativos de la Universidad Nacional de San Antonio Abad del Cusco (periodo 2026-I). El alcance de este documento comprende las tareas realizadas a partir de la gestión de procesos y el cambio de contexto (17 de julio) hasta la implementación de las llamadas al sistema y el shell interactivo (14 de agosto), e incluye el planificador con prioridades, el driver de disco ATA, el sistema de archivos FAT16 de solo lectura y el cargador de ejecutables ELF32. Para cada hito se describen los componentes implementados, las decisiones técnicas relevantes, los problemas encontrados durante la depuración y su solución, con base en la bitácora técnica del proyecto.

Gestión de Procesos y Cambio de Contexto (Tarea del 17 de julio)

Esta tarea introduce la abstracción de proceso dentro del kernel: la estructura de control de proceso (PCB), la reserva de una pila independiente para cada uno y el mecanismo de bajo nivel que permite transferir la ejecución de un proceso a otro sin perder su estado.

Bloque de control de proceso (PCB)

Cada proceso se representa mediante la siguiente estructura, enlazada en una lista circular:

```
typedef struct process {
    uint32_t pid;
    uint32_t state;      /* READY / RUNNING / BLOCKED / DEAD */
    uint32_t priority;
    uint32_t esp;        /* stack pointer guardado */
    uint32_t *stack;     /* base del stack, 16 KB */
    char name[32];
    uint32_t ticks_total;
    uint32_t ticks_quantum;
    struct process *next; /* lista circular */
} process_t;
```

Nota. Estructura del Process Control Block (PCB) definida en process.h.

Pila inicial de un proceso

La pila de cada proceso nuevo se prepara imitando el marco que deja isr.asm antes de restaurar el contexto, de modo que la secuencia pop gs/fs/es/ds, popad e iret funcione sin casos especiales al arrancar el proceso por primera vez:

```
ESP → gs, fs, es, ds      (segmentos)
    EDI, ESI, EBP, ESP,
    EBX, EDX, ECX, EAX    (marco de pushad)
    EIP                    (entry point del proceso)
    CS = 0x10
    EFLAGS = 0x202        (interrupciones habilitadas)
```

Mecanismo de cambio de contexto

El cambio de contexto se realiza dentro del propio manejador de la IRQ0 del timer. La función scheduler_tick recibe el ESP actual del proceso interrumpido y devuelve el ESP del proceso que debe ejecutarse a continuación:

```
irq0:
    pushad
    push ds
    push es
    push fs
    push gs

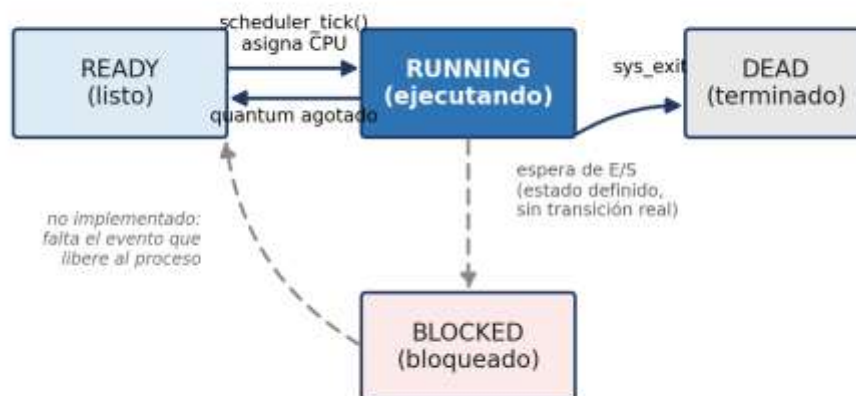
    call timer_handler

    mov al, 0x20
    out 0x20, al      ; EOI antes del cambio

    push esp
    call scheduler_tick ; devuelve el nuevo ESP en EAX
    add esp, 4
    mov esp, eax      ; cargar el stack del siguiente proceso

    pop gs
    pop fs
    pop es
    pop ds
    popad
    iret
```

Figura 1. Diagrama de estados de un proceso en Learnix OS



El planificador omite los procesos en estado BLOCKED o DEAD al recorrer la lista circular.

Nota. El estado BLOCKED está definido en el PCB, pero aún no existe un evento del sistema que saque a un proceso de ese estado.

Resultado obtenido

Figura 2: Gestión de procesos con planificador Round Robin

```

QEMU
Machine View
=== Process Management ===
[PROC] Gestor de procesos inicializado
[SCHED] Scheduler con prioridades inicializado

[PROC] Creado PID=1 [Task-A] prioridad=4 quantum=20
[PROC] Creado PID=2 [Task-B] prioridad=2 quantum=10
[PROC] Creado PID=3 [Task-C] prioridad=1 quantum=5
[PROC] Creado PID=4 [Shell] prioridad=4 quantum=20

[PROC] Lista de procesos:
PID=1 [Task-A] pri=4 q=20 estado=READY
PID=2 [Task-B] pri=2 q=10 estado=READY
PID=3 [Task-C] pri=1 q=5 estado=READY
PID=4 [Shell] pri=4 q=20 estado=READY
  
```

Problemas resueltos

Tabla 1. Problemas resueltos durante la gestión de procesos y el cambio de contexto

Problema	Causa	Solución
“no hay tal instrucción: popad”	GCC ensambla en modo 64 bits y no acepta popad en inline asm	Emitir el opcode directo: .byte 0x61

Problema	Causa	Solución
“referencia a current_process sin definir”	La variable estaba declarada static en process.c	Quitar static para exportarla
Los procesos no alternaban	Los offsets del stack en context_switch.asm no coincidían	Mover el cambio a isr.asm pasando el ESP por argumento
“idt_load inconsistently redefined”	El símbolo estaba declarado como extern y global a la vez	Dejar solo global

Planificador con Prioridades (Tarea del 24 de julio)

Sobre la base de procesos y cambio de contexto se implementó un planificador Round Robin en el que el quantum asignado a cada proceso es proporcional a su prioridad, de modo que los procesos de mayor prioridad reciben más tiempo de CPU por turno sin dejar de ceder el control periódicamente.

Diseño de las prioridades

Tabla 2. Niveles de prioridad y quantum asignado por el planificador

Prioridad	Constante	Valor	Quantum	Tiempo de CPU
Alta	PRIORITY_HIGH	4	20 ticks	200 ms
Normal	PRIORITY_NORMAL	2	10 ticks	100 ms
Baja	PRIORITY_LOW	1	5 ticks	50 ms

El quantum se calcula multiplicando la prioridad por una constante fija:

```
proc->ticks_quantum = priority * 5;
```

Función scheduler_tick

```
uint32_t scheduler_tick(uint32_t current_esp) {
    if (!scheduler_running || !current_process) return current_esp;

    current_process->ticks_total++;
    tick_count++;

    if (tick_count < current_process->ticks_quantum)
        return current_esp; /* quantum no agotado */
    tick_count = 0;

    current_process->esp = current_esp; /* guardar contexto */

    process_t *next = current_process->next;
    while (next->state == PROCESS_DEAD || next->state == PROCESS_BLOCKED)
        next = next->next;
```

```

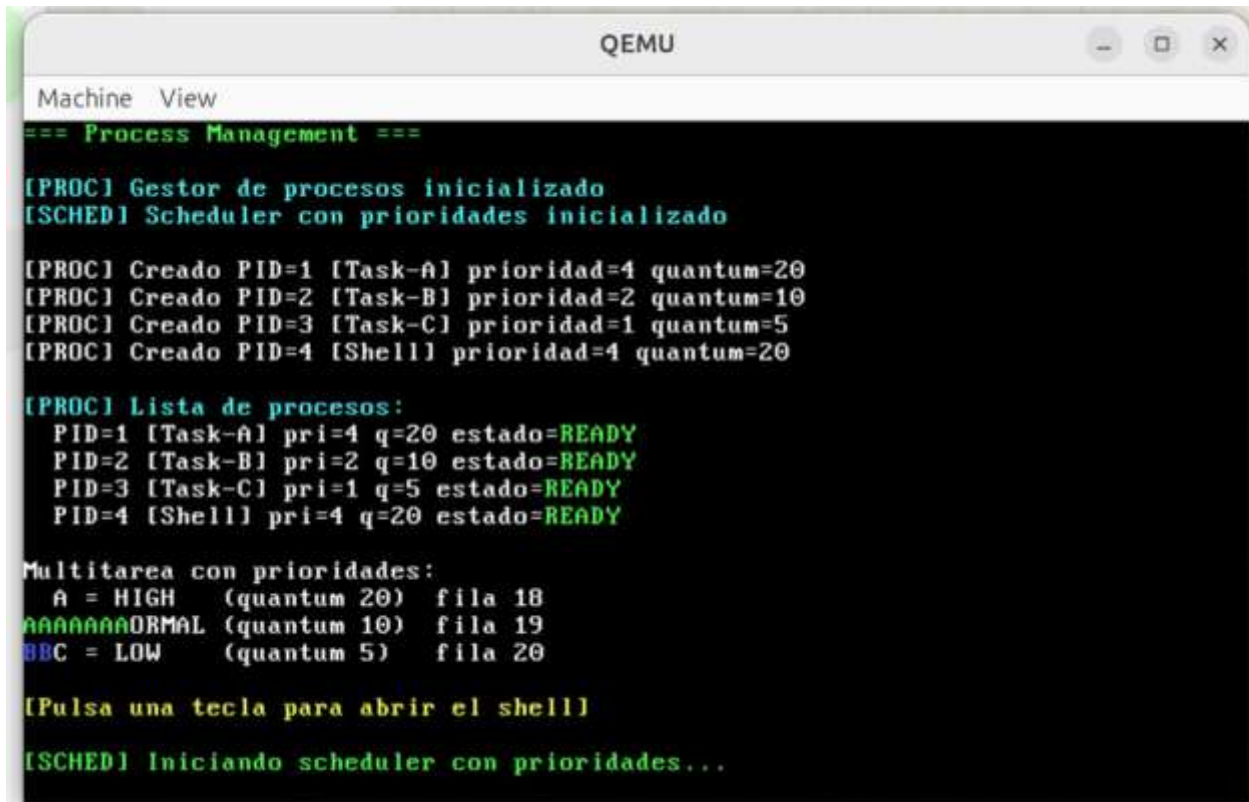
current_process->state = PROCESS_READY;
next->state          = PROCESS_RUNNING;
current_process      = next;

return next->esp;      /* isr.asm lo carga en ESP */
}

```

Resultado Obtenido

Figura 3: Gestión y planificación de procesos



The screenshot shows a QEMU terminal window with the following text:

```

Machine View
=== Process Management ===

[PROC] Gestor de procesos inicializado
[SCHEM] Scheduler con prioridades inicializado

[PROC] Creado PID=1 [Task-A] prioridad=4 quantum=20
[PROC] Creado PID=2 [Task-B] prioridad=2 quantum=10
[PROC] Creado PID=3 [Task-C] prioridad=1 quantum=5
[PROC] Creado PID=4 [Shell] prioridad=4 quantum=20

[PROC] Lista de procesos:
  PID=1 [Task-A] pri=4 q=20 estado=READY
  PID=2 [Task-B] pri=2 q=10 estado=READY
  PID=3 [Task-C] pri=1 q=5 estado=READY
  PID=4 [Shell] pri=4 q=20 estado=READY

Multitarea con prioridades:
  A = HIGH (quantum 20) fila 18
  B = NORMAL (quantum 10) fila 19
  C = LOW (quantum 5) fila 20

[Pulsa una tecla para abrir el shell]

[SCHEM] Iniciando scheduler con prioridades...

```

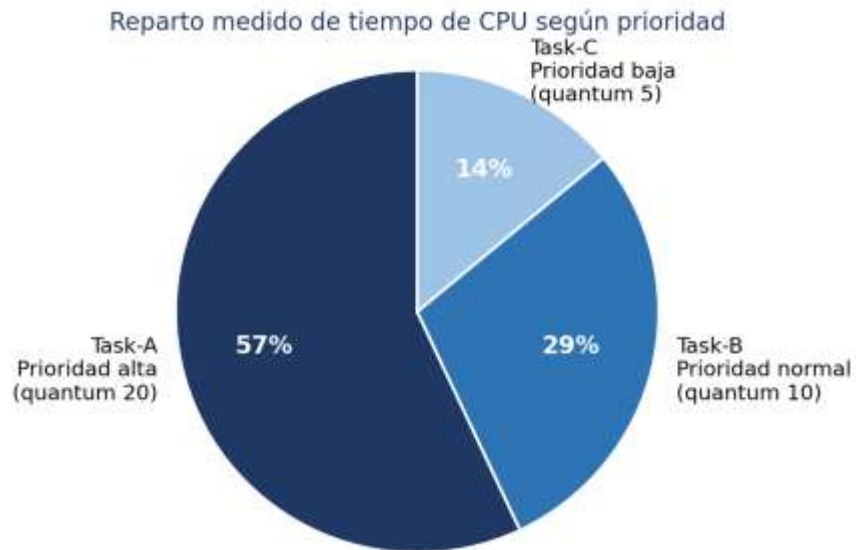
Verificación

La proporción teórica de tiempo de CPU es quantum_i dividido entre la suma de todos los quantums:

- Task-A: $20/35 = 57\%$
- Task-B: $10/35 = 29\%$
- Task-C: $5/35 = 14\%$

Las estadísticas medidas mostraron una relación exacta de 2:1 entre Task-B y Task-C, coincidiendo con la relación 10:5 de sus respectivos quantums.

Figura 4. *Reparto medido de tiempo de CPU entre tres tareas de distinta prioridad*

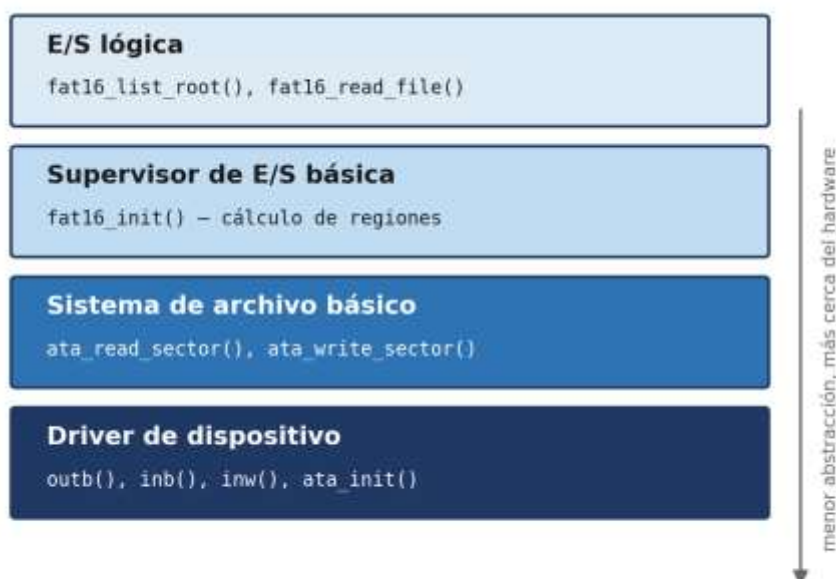


Un detalle relevante de las pruebas visuales es que, con retardos idénticos en las tres tareas, las tres filas se llenaban a la misma velocidad aunque los quantums fueran distintos: el bucle de retardo utilizado tardaba más que un quantum completo, de modo que cada tarea imprimía un carácter por turno sin importar cuánto durara ese turno. Para evidenciar la diferencia entre prioridades es necesario escalar los retardos de forma inversa a la prioridad.

Driver de Disco y Sistema de Archivos FAT16 (Tarea del 31 de julio)

Esta tarea agrega persistencia al sistema mediante un driver de disco ATA en modo PIO con direccionamiento LBA28 y un sistema de archivos FAT16 de solo lectura, organizados siguiendo el modelo de capas típico de los sistemas de gestión de archivos.

Figura 5. *Capas del sistema de gestión de archivos de Learnix OS*



Capas del sistema de archivos

Tabla 3. Correspondencia entre las capas del sistema de archivos y las funciones que las implementan

Capa	Implementación	Archivo
E/S lógica	<code>fat16_list_root()</code> , <code>fat16_read_file()</code>	<code>fat16.c</code>
Supervisor de E/S básica	<code>fat16_init()</code> , cálculo de regiones	<code>fat16.c</code>
Sistema de archivo básico	<code>ata_read_sector()</code> , <code>ata_write_sector()</code>	<code>ata.c</code>
Driver de dispositivo	<code>outb()</code> , <code>inb()</code> , <code>inw()</code> , <code>ata_init()</code>	<code>ata.c</code>

Driver ATA en modo PIO

```
int ata_read_sector(uint32_t lba, uint8_t *buffer) {
    if (ata_wait_ready() < 0) return -1;

    outb(0x1F2, 1);          /* 1 sector */
    outb(0x1F3, (lba & 0xFF)); /* LBA bits 0-7 */
    outb(0x1F4, (lba >> 8) & 0xFF); /* LBA bits 8-15 */
    outb(0x1F5, (lba >> 16) & 0xFF); /* LBA bits 16-23 */
    outb(0x1F6, 0xF0 | ((lba >> 24) & 0x0F)); /* Slave + 24-27 */
    outb(0x1F7, 0x20);        /* READ SECTORS */

    if (ata_wait_drq() < 0) return -1;

    uint16_t *buf16 = (uint16_t *) buffer;
    for (int i = 0; i < 256; i++)
        buf16[i] = inw(0x1F0); /* 256 x 16 b = 512 B */
}
```

```
return 0;
}
```

Decisión clave: selección del disco Slave

GRUB ocupa el Master del bus IDE primario con la propia ISO montada como CD-ROM, de modo que disk.img debe seleccionarse como Slave del mismo bus:

```
outb(ATA_PRIMARY_DRIVE, 0xB0); /* en ata_init: 0xA0 = Master, 0xB0 = Slave */
outb(ATA_PRIMARY_DRIVE, 0xF0 | ((lba >> 24) & 0x0F)); /* en las lecturas */
```

Con 0xA0 el driver leía el sector 0 del CD-ROM en lugar del disco virtual y fat16_init() fallaba con el mensaje “No se pudo leer el boot sector”.

Estructura FAT16 del disco de 16 MB

Sector 0	Sector 4	Sector 68	Sector 100
+- Boot Sector +- FAT 1 + 2 +- Root Dir ---+ Área de datos ->			
(1 sector)	(64 sect.)	(32 sect.)	(clusters 2, 3, ...)

Figura 6. Distribución de regiones en el disco virtual FAT16 de 16 MB



El cálculo de las regiones se realiza en fat16_init():

```
fat_start = reserved_sectors; /* 4 */
root_start = fat_start + (fat_count * sectors_per_fat); /* 68 */
data_start = root_start + (root_entries * 32 + 511) / 512; /* 100 */
```

y el sector físico correspondiente a un clúster se obtiene con:

```
sector = data_start + (cluster - 2) * sectors_per_cluster
```

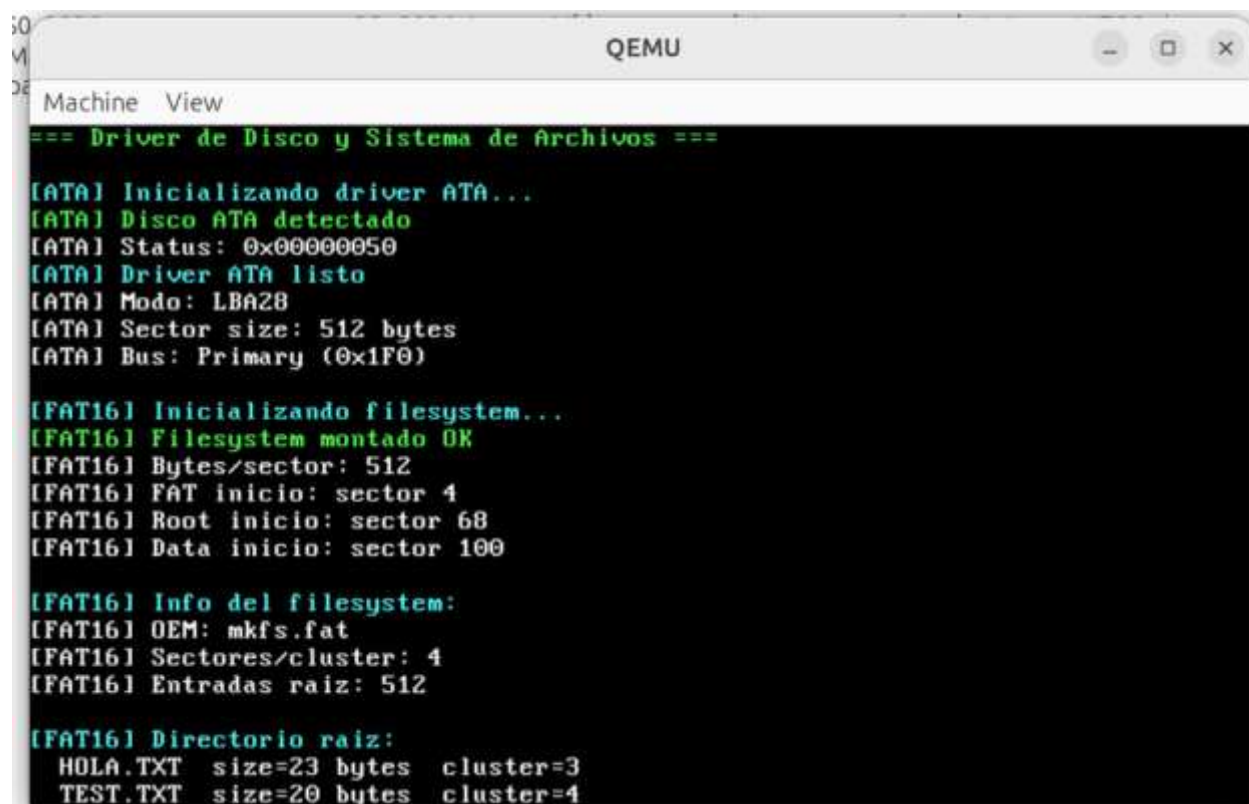
Formato de nombres de archivo

Las entradas de directorio almacenan el nombre en 8 bytes más 3 de extensión, rellenos con espacios y sin punto (por ejemplo, “HOLA.TXT” se guarda como “HOLA TXT”). La función

fat16_read_file() reconstruye ese formato de 11 caracteres antes de comparar, en lugar de comparar campo por campo.

Resultados de la verificación

Figura 7 Learnix en ejecución sobre ATA y sistema de archivos FAT 16



```

Machine View
=== Driver de Disco y Sistema de Archivos ===

[ATA] Inicializando driver ATA...
[ATA] Disco ATA detectado
[ATA] Status: 0x00000050
[ATA] Driver ATA listo
[ATA] Modo: LBA28
[ATA] Sector size: 512 bytes
[ATA] Bus: Primary (0x1F0)

[FAT16] Inicializando filesystem...
[FAT16] Filesystem montado OK
[FAT16] Bytes/sector: 512
[FAT16] FAT inicio: sector 4
[FAT16] Root inicio: sector 68
[FAT16] Data inicio: sector 100

[FAT16] Info del filesystem:
[FAT16] OEM: mkfs.fat
[FAT16] Sectores/cluster: 4
[FAT16] Entradas raiz: 512

[FAT16] Directorio raiz:
HOLA.TXT size=23 bytes cluster=3
TEST.TXT size=20 bytes cluster=4
  
```

El estado 0x50 corresponde a los bits DRDY (bit 6) y DSC (bit 4) activos, lo que indica que el disco está listo y la cabeza posicionada.

Funciones de administración de archivos

Tabla 4. Funciones de administración de archivos implementadas en Learnix OS

Función	Implementada	Observación
Identificar y ubicar un archivo	Sí	Búsqueda por nombre en el directorio raíz
Directorio con ubicación y atributos	Sí	fat16_entry_t con nombre, tamaño y clúster
Control de acceso de usuario	No	No existen usuarios ni permisos
Bloqueo de archivos	No	Sin exclusión mutua
Asignar archivos a bloques libres	No	El sistema de archivos es de solo lectura

Función	Implementada	Observación
Administrar espacio libre	No	No se gestiona la FAT para escritura

Cargador de Ejecutables ELF (Tarea del 7 de agosto)

Con el sistema de archivos disponible, esta tarea añade la capacidad de cargar y ejecutar programas de usuario en formato ELF32 estático, integrando el sistema de archivos, la gestión de memoria y la gestión de procesos desarrollados en los hitos anteriores.

Estructuras del formato ELF

```
typedef struct { /* ELF Header - 52 bytes */
    uint8_t e_ident[16]; /* magic 0x7F 'E' 'L' 'F' + clase + endianness */
    uint16_t e_type; /* 2 = ET_EXEC */
    uint16_t e_machine; /* 3 = EM_386 */
    uint32_t e_version;
    uint32_t e_entry; /* punto de entrada */
    uint32_t e_phoff; /* offset de la tabla de Program Headers */
    uint32_t e_shoff;
    uint32_t e_flags;
    uint16_t e_ehsize;
    uint16_t e_phentsize;
    uint16_t e_phnum; /* número de Program Headers */
    uint16_t e_shentsize;
    uint16_t e_shnum;
    uint16_t e_shstrndx;
} __attribute__((packed)) elf32_hdr_t;

typedef struct { /* Program Header - 32 bytes */
    uint32_t p_type; /* 1 = PT_LOAD */
    uint32_t p_offset; /* offset en el archivo */
    uint32_t p_vaddr; /* dirección de destino */
    uint32_t p_paddr;
    uint32_t p_filesz; /* bytes en el archivo */
    uint32_t p_memsz; /* bytes en memoria */
    uint32_t p_flags; /* R / W / X */
    uint32_t p_align;
} __attribute__((packed)) elf32_phdr_t;
```

Proceso de carga

- fat16_read_file() lee el ELF completo en un búfer de 64 KB
- elf_verify() comprueba el magic, la clase de 32 bits, ET_EXEC y EM_386
- Para cada segmento PT_LOAD se copian p_filesz bytes a p_vaddr
- Se rellenan con ceros los p_memsz - p_filesz bytes restantes (la sección .bss)
- Finalmente se invoca process_create(e_entry, nombre, PRIORITY_NORMAL)

```

for (int i = 0; i < hdr->e_phnum; i++) {
    if (phdrs[i].p_type != PT_LOAD) continue;

    uint8_t *dest = (uint8_t *) phdrs[i].p_vaddr;
    uint8_t *src = elf_buffer + phdrs[i].p_offset;

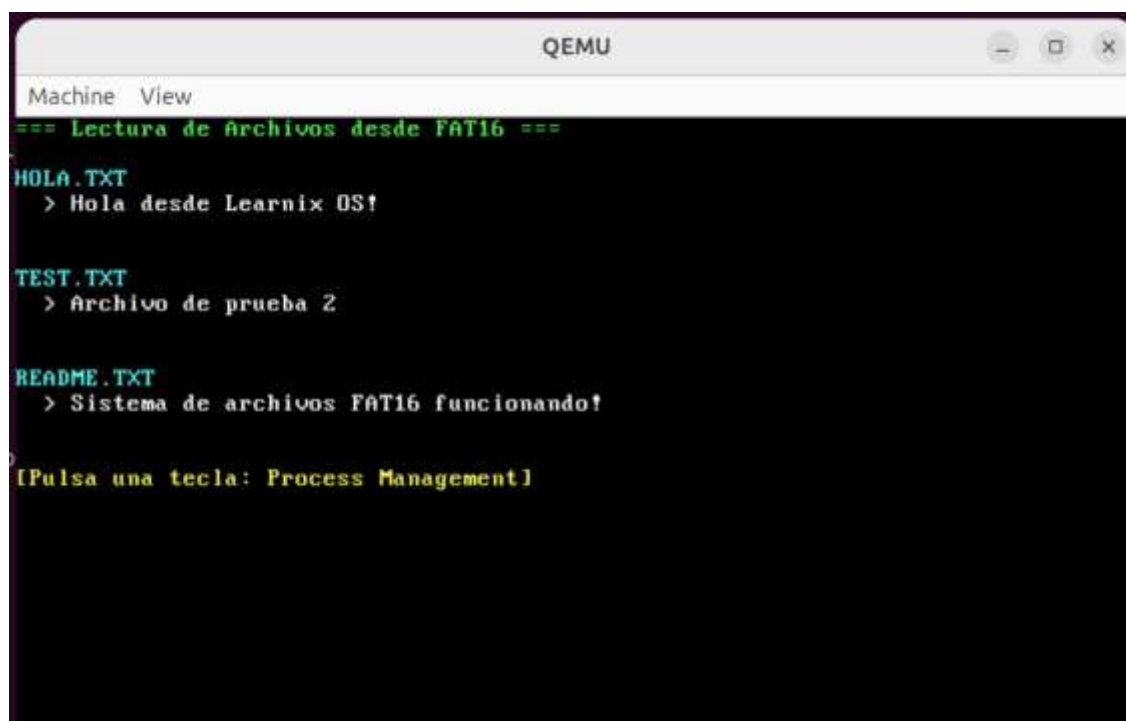
    for (uint32_t b = 0; b < phdrs[i].p_filesz; b++)
        dest[b] = src[b];

    for (uint32_t b = phdrs[i].p_filesz; b < phdrs[i].p_memsz; b++)
        dest[b] = 0;      /* .bss */
}

```

Resultado obtenido

Figura 8: Lectura de archivos desde FAT 26



Problema principal: segmentos fuera del área mapeada

Al compilar con `-Wl,-Ttext=0x400000`, GCC generaba cinco segmentos `PT_LOAD`, y el último quedaba ubicado en `0x08048134`, muy por encima de los 4 MB que mapea la paginación identity map descrita en el hito de gestión de memoria. Al intentar copiar ese segmento el sistema se colgaba con un fallo de página:

```

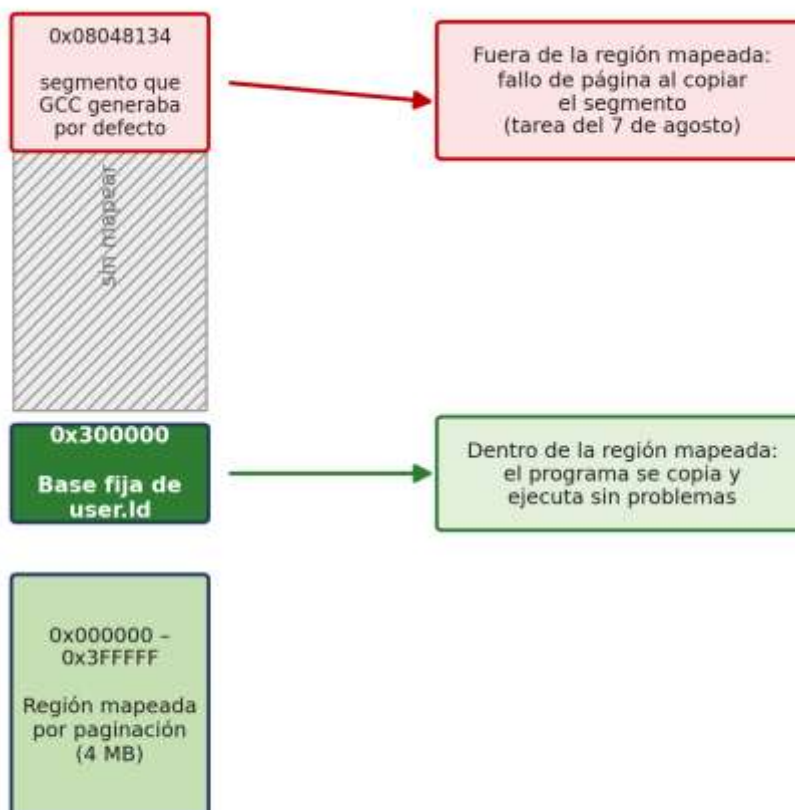
LOAD 0x000000 0x002ff000 ... R
LOAD 0x001000 0x00300000 ... R E
LOAD 0x002000 0x00301000 ... R

```



```
LOAD 0x002ff4 0x00302ff4 ... RW
LOAD 0x003134 0x08048134 ... R <- fuera de los 4 MB mapeados
```

Figura 9. Mapa de memoria de 4 MB y el segmento generado fuera del área mapeada



La solución consistió en escribir un linker script propio, `user.ld`, que fija la dirección de carga en `0x300000` y descarta las secciones de notas y metadatos que GCC agrega por defecto:

```
ENTRY(_start)

SECTIONS
{
    . = 0x300000;
    .text : { *(.text) *(.text.*) }
    .rodata : { *(.rodata) *(.rodata.*) }
    .data : { *(.data) *(.data.*) }
    .bss : { *(.bss) *(.bss.*) *(COMMON) }

    /DISCARD/ : { *(.note*) *(.comment) *(.eh_frame*) }
}
```

Con el script, el binario resultante contiene un único segmento cargable:

```
LOAD 0x001000 0x00300000 0x0030c 0x0030c RWE 0x1000 <- un solo segmento
```

El comando de compilación definitivo para los programas de usuario quedó como:

```
gcc -m32 -ffreestanding -fno-stack-protector -fno-builtin \
-nostdlib -nostartfiles -static -no-pie \
-T user.ld -o suma.elf suma.c
```

Otro problema: orden de inicialización

La función `elf_load()` invoca `process_create()`, que inserta el nuevo PCB en la lista circular de procesos. Si esta llamada ocurre antes de `process_init()`, la lista queda corrupta y el gestor de procesos no arranca. El orden correcto establecido en `kernel.c` es:

```
process_init();      /* primero inicializar la lista */
scheduler_init();
process_create(...); /* procesos del kernel */
elf_load("HELLO ELF"); /* después cargar ejecutables */
process_print_all();
scheduler_start();
```

Restricciones de los programas de usuario

- Sin biblioteca estándar de C: toda la salida se escribe directamente en 0xB8000
- El punto de entrada debe llamarse `_start`
- Enlazado estático (-static); un ELF dinámico requeriría un enlazador dinámico propio
- Deben terminar en un bucle `hlt` para no consumir CPU innecesariamente

Llamadas al Sistema y Shell (Tarea del 14 de agosto)

El último hito cubierto en este informe cierra el ciclo del sistema operativo: expone los servicios del kernel a los programas de usuario mediante una interfaz de llamadas al sistema basada en INT 0x80, y ofrece una interfaz de usuario mediante un shell interactivo.

Mecanismo INT 0x80

La convención de registros utilizada para invocar servicios del kernel es la siguiente:

Tabla 5. Convención de registros para invocar una llamada al sistema

Registro	Uso
EAX	Número de syscall / valor de retorno
EBX	Argumento 1
ECX	Argumento 2
EDX	Argumento 3

La invocación desde código en espacio de usuario se realiza mediante una función inline:

```
static inline uint32_t syscall(uint32_t num, uint32_t a1,
                             uint32_t a2, uint32_t a3) {
    uint32_t ret;
    __asm__ volatile ("int $0x80"
        : "=a"(ret)
        : "a"(num), "b"(a1), "c"(a2), "d"(a3)
        : "memory");
    return ret;
}
```

y el manejador correspondiente en `isr.asm` guarda los segmentos, restablece el selector del kernel y despacha la llamada a `syscall_handler`:

```
int80_handler:
    push ds
    push es
    push fs
    push gs

    mov ax, 0x10
    mov ds, ax
    mov es, ax

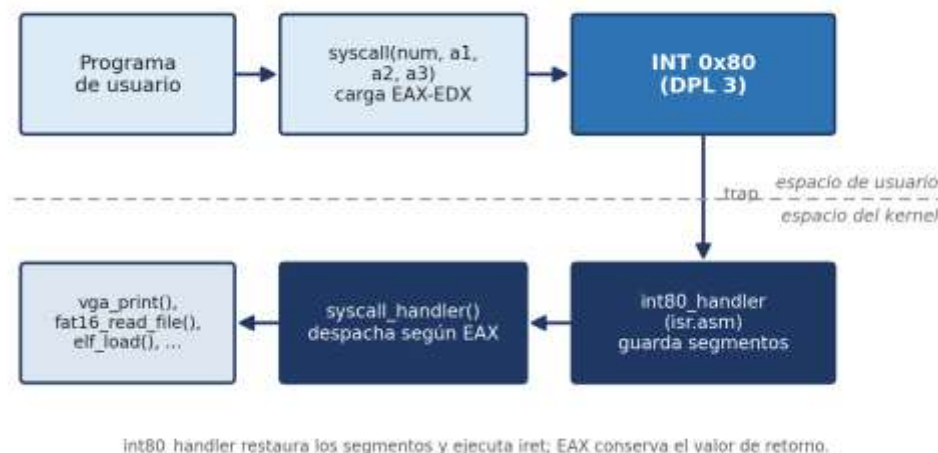
    push edx      ; argumentos en orden inverso (cdecl)
    push ecx
    push ebx
    push eax
    call syscall_handler
    add esp, 16    ; EAX conserva el valor de retorno

    pop gs
    pop fs
    pop es
    pop ds
    iret
```

Este manejador se registra en la IDT con DPL 3, requisito indispensable para que pueda invocarse desde el modo usuario:

```
idt_set_gate(0x80, (uint32_t)int80_handler, 0x10, 0xEE);
```

Figura 10. Flujo de una llamada al sistema a través de `INT 0x80`



Llamadas al sistema implementadas

Tabla 6. Llamadas al sistema implementadas en Learnix OS

EAX	Nombre	Argumentos	Backend	Hito
1	sys_exit	EBX = código	process_exit()	17/07
2	sys_write	EBX = buffer	vga_print()	Hito 1
3	sys_read	EBX = buffer, ECX = máx	esperar_tecla_char()	Hito 1
4	sys_open	EBX = nombre	fat16_read_file()	31/07
5	sys_close	EBX = fd	—	—
6	sys_fread	EBX / ECX / EDX	fat16_read_file()	31/07
7	sys_exec	EBX = nombre ELF	elf_load()	07/08
8	sys_list	—	fat16_list_root()	31/07

Shell interactivo

El shell ofrece seis comandos: ls, cat <archivo>, run <elf>, ps, help y exit. La conversión de los nombres escritos en minúscula al formato 8+3 de FAT16 se realiza en el propio shell, lo que permite escribir, por ejemplo, cat hola.txt:

```
static void to_fat_name(const char *in, char *out) {
    for (int i = 0; i < 11; i++) out[i] = ' ';
    out[11] = '\0';

    int i = 0;
    while (in[i] && in[i] != ' ' && i < 8) {
        char c = in[i];
```

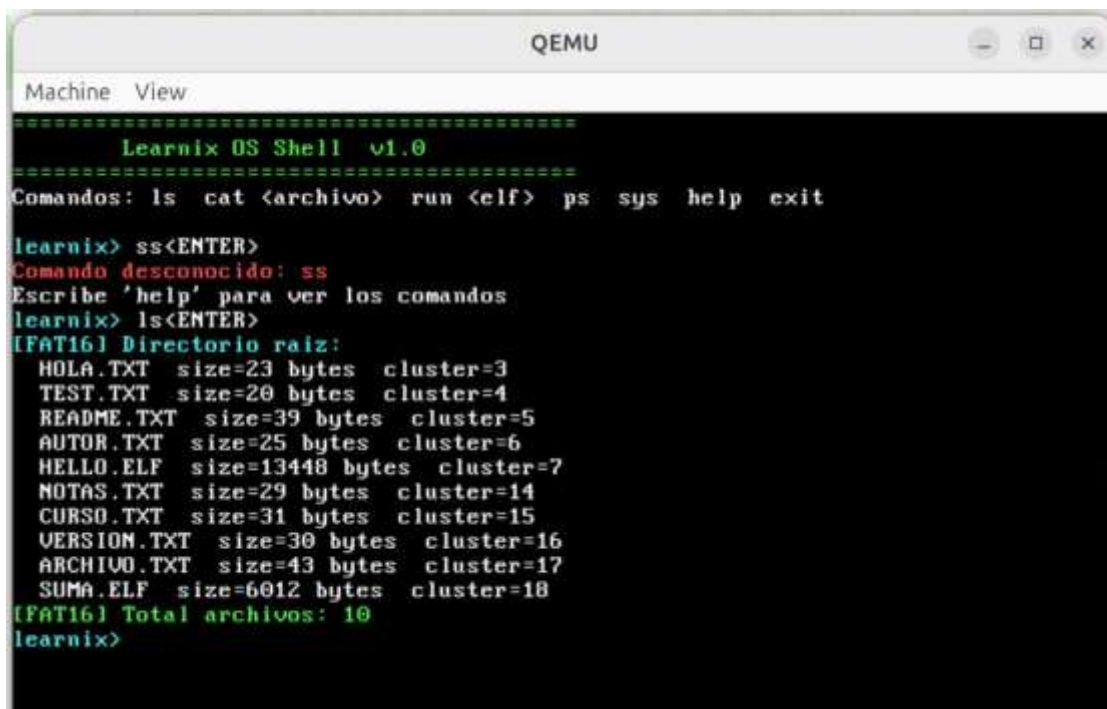
```

    if (c >= 'a' && c <= 'z') c -= 32;
    out[i++] = c;
}
while (in[i] && in[i] != '.') i++;
if (in[i] == '.') {
    i++;
    int j = 8;
    while (in[i] && j < 11) {
        char c = in[i++];
        if (c >= 'a' && c <= 'z') c -= 32;
        out[j++] = c;
    }
}
}
}

```

Resultado obtenido

Figura 11: Visualizar desde el Shell con 7 comandos disponibles



```

=====
Learnix OS Shell v1.0
=====
Comandos: ls cat <archivo> run <elf> ps sys help exit

learnix> ss<ENTER>
Comando desconocido: ss
Escribe 'help' para ver los comandos
learnix> ls<ENTER>
[FAT16] Directorio raiz:
HOLA.TXT size=23 bytes cluster=3
TEST.TXT size=20 bytes cluster=4
README.TXT size=39 bytes cluster=5
AUTOR.TXT size=25 bytes cluster=6
HELLO.ELF size=13448 bytes cluster=7
NOTAS.TXT size=29 bytes cluster=14
CURSO.TXT size=31 bytes cluster=15
VERSION.TXT size=30 bytes cluster=16
ARCHIVO.TXT size=43 bytes cluster=17
SUMA.ELF size=6012 bytes cluster=18
[FAT16] Total archivos: 10
learnix>

```

Problema 1: el teclado no respondía

Con la IRQ1 habilitada, keyboard_handler() leía el puerto 0x60 en cuanto llegaba la interrupción y vaciaba el búfer del controlador; cuando esperar_tecla_char() intentaba hacer polling, ya no quedaba ningún scancode disponible. La solución fue enmascarar la IRQ1 y dejar el teclado por polling puro:

```

outb(0x21, 0xFE); /* antes: 0xFC - ahora solo IRQ0 habilitada */
char esperar_tecla_char() {
    uint8_t sc;
    while (1) {
        if (!(inb(0x64) & 0x01)) continue; /* ¿hay datos? */
        sc = inb(0x60);
        if (sc & 0x80) continue; /* tecla soltada */
        if (sc >= 128) continue;

        char c = scancode_map[sc];
        if (c == 0) continue; /* shift, ctrl, ... */
        if (c == '\n') return '\n';
        if (c == '\b') return '\b';

        char str[2] = { c, '\0' };
        vga_print(str); /* eco en pantalla */
        return c;
    }
}

```

Problema 2: todo se escribía en la esquina de la pantalla

timer_handler() imprimía el contador de ticks con vga_print_int(), función que escribe en la posición actual del cursor y lo deja ahí. Como el timer se dispara 100 veces por segundo, el cursor quedaba fijo en la esquina superior derecha y el shell terminaba escribiendo en ese punto. La solución fue eliminar toda impresión dentro del manejador del timer:

```

void timer_handler() {
    timer_ticks++;
    /* Sin impresión: el shell necesita el cursor libre */
}

```

Problema 3: ls y cat colgaban el sistema

El driver ATA funciona por polling; si el timer interrumpe a mitad de la transferencia de un sector, el estado del controlador se pierde y la lectura nunca termina. La solución fue deshabilitar las interrupciones durante el acceso a disco:

```

__asm__ volatile ("cli");
fat16_list_root();
__asm__ volatile ("sti");

```

Problema 4: las tareas de fondo bloqueaban el shell

Con Task-A y Task-B activas junto al shell, sus bucles de retardo consumían el CPU y el shell no llegaba a leer el teclado a tiempo. Se optó por dejar únicamente el shell en ejecución y añadir el comando ps para inspeccionar los procesos activos cuando fuera necesario.

Modelo de Entrada/Salida en Learnix OS

Toda la entrada/salida del sistema es síncrona y bloqueante: no existe acceso directo a memoria (DMA), ni entrada/salida asíncrona, ni mecanismos de notificación mediante callbacks.

Tabla 7. Mecanismos de entrada/salida y su estado de implementación

Mecanismo	Estado
PIO / polling	Implementado — es el modelo usado en todo el sistema
DMA	No implementado
E/S dirigida por interrupciones	Solo para el timer
E/S asíncrona con callbacks	No implementado

El único componente no bloqueante del sistema es el planificador, que se activa desde la IRQ0 del timer en lugar de recurrir a polling. Un ejemplo representativo del patrón bloqueante empleado en todo el driver de disco es la espera activa de ata.c:

```
static int ata_wait_ready() {
    uint32_t timeout = 100000;
    while (timeout-->0) {
        uint8_t status = inb(ATA_PRIMARY_STATUS);
        if (status & ATA_STATUS_ERR) return -1;
        if (!(status & ATA_STATUS_BSY) && (status & ATA_STATUS_DRDY))
            return 0;
        /* el CPU no hace nada más mientras espera */
    }
    return -1;
}
```

Resumen General de Problemas y Soluciones

La siguiente tabla consolida, en orden cronológico, los problemas de depuración enfrentados desde la gestión de procesos hasta el shell interactivo, junto con su causa raíz y la solución aplicada.

Tabla 8. Resumen cronológico de problemas de depuración y sus soluciones

#	Problema	Causa	Solución
1	Las IRQ no llegaban a los manejadores	GRUB deja el PIC en vectores distintos a los habituales	Registrar en 0x08 y 0x0E, con selector 0x10
2	“no hay tal instrucción: popad”	GCC ensambla en modo 64 bits	Emitir el opcode directo: .byte 0x61
3	“referencia a current_process sin definir”	Variable declarada static	Quitar static en process.c
4	Los procesos no alternaban	Offsets incorrectos en context_switch.asm	Pasar el ESP como argumento a scheduler_tick

#	Problema	Causa	Solución
5	“idt_load inconsistently redefined”	Símbolo declarado como extern y global	Dejar solo global
6	“This is not a bootable disk”	QEMU arrancaba desde el disco duro	Añadir -boot order=d
7	“No se pudo leer el boot sector”	El driver leía el CD-ROM (Master)	Seleccionar el Slave: 0xB0 / 0xF0
8	cat no encontraba archivos	Comparación de nombres incorrecta	Reconstruir el nombre 8+3 completo
9	Sistema colgado al cargar un ELF	Segmento en 0x08048134, fuera de los 4 MB	Linker script user.ld con base 0x300000
10	Lista de procesos corrupta	elf_load() antes de process_init()	Reordenar la inicialización
11	El teclado no respondía	La IRQ1 consumía los scancodes	Enmascarar IRQ1: outb(0x21, 0xFE)
12	Texto en la esquina de la pantalla	timer_handler() movía el cursor	Quitar la impresión del timer
13	ls y cat colgaban el sistema	El timer interrumpía el polling ATA	cli / sti alrededor del acceso a disco
14	El shell no respondía con tareas activas	Los bucles de retardo consumían el CPU	Ejecutar solo el shell
15	“Failed to get ‘write’ lock”	disk.img seguía montado	sudo umount /tmp/mnt_learnix

Estado del Proyecto

Implementado

- Procesos con PCB y cambio de contexto completo
- Planificador Round Robin con tres niveles de prioridad
- Driver ATA en modo PIO con direccionamiento LBA28
- Sistema de archivos FAT16 en modo lectura
- Cargador de ejecutables ELF32 estáticos
- Llamadas al sistema mediante INT 0x80
- Shell interactivo con seis comandos

Pendiente

- Escritura en FAT16 (crear, borrar y modificar archivos)

- Gestión de espacio libre en la FAT
- Separación real entre anillo 0 y anillo 3 (TSS)
- Espacios de direcciones independientes por proceso
- Control de acceso y permisos de archivos
- Bloqueo de archivos y primitivas de sincronización
- DMA y entrada/salida asíncrona
- Ejecutables dinámicos con bibliotecas compartidas

Conclusiones

El desarrollo cubierto en este informe consolidó a Learnix OS como un sistema multitarea funcional: sobre la base de memoria física y paginación construida en el hito anterior, se añadieron procesos con cambio de contexto real, un planificador con prioridades verificable empíricamente, persistencia en disco mediante FAT16, carga de ejecutables ELF32 y una interfaz de llamadas al sistema que permite a los programas de usuario interactuar con el kernel a través de un shell.

La mayoría de los problemas encontrados no se originaron en errores de lógica de alto nivel, sino en discrepancias entre el entorno asumido por las herramientas de compilación estándar (GCC, GDB, direcciones de carga por defecto) y las restricciones reales del entorno bare-metal construido, como el mapeo de solo 4 MB de memoria o los vectores de interrupción no convencionales que deja GRUB. Esto refuerza la importancia de verificar cada supuesto de la toolchain contra el comportamiento observado en el emulador, en lugar de asumir el comportamiento típico de un sistema operativo de propósito general.

El sistema resultante cubre el ciclo completo de un sistema operativo mínimo: inicialización de hardware, gestión de memoria, multitarea, persistencia y una interfaz de usuario. El trabajo pendiente, principalmente la escritura en FAT16, la separación de anillos de privilegio y los espacios de direcciones independientes por proceso, define una ruta clara para evolucionar Learnix OS hacia un sistema con mayor aislamiento y robustez.

Bibliografía

Committee, T. I. (1995). *System V Application Binary Interface*.

Intel Corporation. (2023). *Intel 64 and IA-32 Architectures Software Developer's Manual (Vol. 3A)*.

Microsoft Corporation. (2000). *FAT32 File System Specification*.

OSDev Wiki. (s.f.). *OSDev Wiki*. Obtenido de <https://wiki.osdev.org>

Patterson, D. A. (2017). *Organización y diseño de computadores*. Elsevier. Obtenido de *Organización y diseño de computadores*.

Sagi21805. (s.f.). Obtenido de The LearnixOS Book: <https://www.learnix-os.com>

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Fundamentos de sistemas operativos*. McGraw-Hill.

Tanenbaum, A. S., & Bos, H. (2015). *Sistemas operativos modernos*. Pearson.